

README - SAé 3.01 - Développement d'un jeu de cartes en TypeScript

Présenté par : Uzeyir Unver et Arona

Ce document explique le fonctionnement et les choix que nous avons faits pour notre système de gestion de cartes. Nous décrivons comment chaque partie du programme fonctionne et pourquoi nous avons pris ces décisions, pour montrer notre maîtrise des concepts demandés dans la SAé.

1. Architecture Générale (Approche Objet)

Nous avons restructuré l'application autour de classes interagissant entre elles pour respecter le paradigme objet.

- **Classe Carte** : Représente l'élément de base, immuable après création.
- **Classe Joueur** : Encapsule la main du joueur, son score et ses actions.
- **Classe Abstraite Jeu** : Définit les fonctions commun à tous les jeux (gestion des tours, mélange, distribution, sauvegarde).
- **Classes Filles (Bataille, Belote)** fonction des jeu implémentent uniquement les règles spécifiques (conditions de victoire.,etc).

2. Encapsulation et Sécurité des Données

Afin de garantir l'intégrité des données, nous avons appliqué une encapsulation stricte :

- **Attributs Privés** : Les propriétés valeur, couleur (dans Carte) ou score, main sont en private.
- **Accesseurs (Getters)** : L'accès aux données se fait uniquement via des méthodes dédiées (getValeur(), getScore()), empêchant toute modification accidentelle depuis l'extérieur.

3. Gestion des Couleurs (Enum)

Comme dans la version précédente, nous utilisons une enum Couleur (Cœur, Carreau, Trèfle, Pique). Cela évite les erreurs de frappe, tout en facilitant l'ajout futur de couleurs

4.Héritage

Méthodes Communes (Factorisation)

Les fonctions communes des jeux sont codées dans la classe mère :

- **Mélange** : Utilisation de l'algorithme de Fisher-Yates pour un mélange parfait.
- **Distribution** : Distribution carte par carte aux joueurs .

Méthodes Abstraites

Les méthodes dépendantes des règles sont déclarées abstraites.

- `comparerCartes()` : La Bataille compare les indices dans un tableau, la Belotte gère l'Atout.
- `calculerPoints()` : La Bataille compte 1 point par pli, la Belotte additionne les valeurs des cartes.

5. Fonctionnalités Avancées : Sauvegarde et Replay

Système de Replay

- Nous utilisons un attribut protégé historique: `string[]` dans la classe `Jeu`.
- Chaque action (pli remporté, distribution) est enregistrée via `ajouterAction()`.
- La méthode `afficherReplay()` permet de relire l'intégralité de la partie en fin de jeu.

Sauvegarde

- **Sauvegarde** : L'état du jeu (Joueurs, Mains, Scores, Historique) est converti en chaîne JSON via `JSON.stringify`.

6. Implémentation des Jeux Spécifiques

La Bataille

- Utilise un tableau ordonné de "2" à "As".
- La comparaison se fait sur l'index de la valeur dans ce tableau.

La Belotte

- Définit une couleur d'Atout
- Gère deux ordres de valeurs différents selon si la carte est un Atout ou non.
- Implémente le calcul des points spécifique (Valet Atout = 20, As = 11, etc.).

7. Scénario de Test

Le programme principal démontre test le jeu :

1. Nouvelle partie
2. Lancement automatique de la partie (jouerPartie()).
3. Affichage du déroulement complet via le Replay.
4. Génération de la chaîne de sauvegarde pour prouver la persistance.